สรุประบบหลังบ้าน (Backend Summary)

ระบบจัดการคำสั่งซื้อและตรวจสอบคลังสินค้า ร้านวัสดุก่อสร้างอำพรคอนกรีต (SOML)

นายตรัยรัตน์ วงษ์สิทธิ์ 67543210028-6 · นายพนาวุฒน์ อภิปสันติ 67543210040-1 · 22/09/2569

รายการ	รายละเอียด
ขอบเขตของเอกสาร	อธิบายเฉพาะระบบหลังบ้าน (Backend) ว่าแบ่งส่วนอย่างไร แต่ละส่วนรับผิดชอบอะไร และกลไกสำคัญทำงานอย่างไร
ที่มาของเนื้อหา	ปริญญานิพนธ์บทที่ 3 ฉบับตรวจแก้ (ตารางที่ 3.13, 3.14, 3.17–3.20 และหัวข้อ 3.7.3) และซอร์สโค้ดจริงในโฟลเดอร์ soml/backend
สถานะของระบบ	พัฒนาครบทุกโมดูล ทดสอบอัตโนมัติ 130 ข้อผ่านทั้งหมด (13 ชุดทดสอบ)
เทคโนโลยี	Node.js 20 + Express 5 · MySQL 8.0 (InnoDB) · ws (WebSocket) · jsonwebtoken · bcryptjs

1. หลังบ้านรับผิดชอบอะไร

หลังบ้านเป็นศูนย์กลางของตรรกะและสิทธิ์ทั้งหมดของระบบ หน้าจอผู้ใช้ (React) ไม่มีการตัดสินใจทางธุรกิจใด ๆ ทำหน้าที่เพียงแสดงผลและส่งคำขอ ดังนั้นแม้ผู้ใช้จะแก้ไขหน้าจอหรือเรียกเส้นทางบริการโดยตรง ระบบก็ยังบังคับกฎได้ครบ

หลังบ้านทำ	หลังบ้านไม่ทำ
ตรวจตัวตนและสิทธิ์ของทุกคำขอก่อนประมวลผล	ไม่ตัดสินใจแทนผู้ใช้ เช่น ไม่ยืนยันรับเงินเองและไม่ยืนยันจ่ายสินค้าเอง
คำนวณยอดเงินจากราคาในฐานข้อมูลเสมอ ไม่รับราคาจากหน้าจอ	ไม่ติดต่อธนาคาร ไม่มีการตรวจสอบเงินเข้าอัตโนมัติ
ควบคุมสถานะคำสั่งซื้อและตัดสต็อกภายในธุรกรรมเดียว	ไม่สั่งงานเครื่องพิมพ์โดยตรง (ส่งต่อให้บริการพิมพ์ซึ่งเป็นอีกหน่วยติดตั้ง)
ผลักเหตุการณ์ไปยังหน้าจอที่เปิดค้างอยู่ และสร้างการแจ้งเตือนเอง	ไม่เก็บสถานะการทำงานไว้ในหน่วยความจำของตัวเอง ทุกอย่างอ่านจากฐานข้อมูล
บันทึกประวัติทุกเหตุการณ์สำคัญพร้อมผู้กระทำ เวลา และหมายเลขไอพี	ไม่มีเส้นทางใดที่แก้ไขหรือลบประวัติได้

2. โครงสร้างของหลังบ้าน

หลักการที่ยึดคือ **หนึ่งโมดูลในตารางที่ 3.13 เท่ากับหนึ่งโฟลเดอร์** เปิดโฟลเดอร์แล้วต้องรู้ทันทีว่าตรงกับโมดูลใดในเล่ม โมดูลแต่ละตัวมีเอกสารกำกับหัวไฟล์ว่ารับผิดชอบเรื่องเดียวคืออะไร และอ้างถึงหัวข้อใดในปริญญานิพนธ์

2.1 โมดูลหลัก 10 ตัว (ตารางที่ 3.13)

#	โฟลเดอร์	รับผิดชอบเรื่องเดียว	บรรทัด	UC / FR
1	modules/auth	ตรวจรหัสผ่าน ออกโทเคนที่ระบุบทบาท และคืนข้อมูลผู้ใช้ปัจจุบัน	56	UC-01 · FR-01
2	modules/product	ข้อมูลสินค้า หมวดหมู่ ราคา จุดสั่งซื้อเพิ่ม และการปรับสต็อกที่คนเป็นผู้ทำ	176	UC-02 · FR-02
3	modules/order	คำนวณยอด ออกเลขที่ไม่ซ้ำ บันทึกคำสั่งซื้อ และสร้างรหัส QR	175	UC-03, UC-04 · FR-03
4	modules/queue	คิวรอจ่ายสินค้า การเปลี่ยนสถานะ และช่องทางค้างที่ผลักเหตุการณ์	135	UC-05, UC-06 · FR-04
5	modules/dispatch	รับคำขอยืนยันจ่ายสินค้า เปิดธุรกรรม และแปลงผลเป็นคำตอบ	37	UC-07 · FR-05
6	modules/stock-deduction	ธุรกรรมตัดสต็อกแบบอะตอมมิกตามหัวข้อ 3.7.3	74	UC-07 · FR-06
7	modules/	คำนวณตัวชี้วัดสดจากฐานข้อมูลสองระดับ	66	UC-08, UC-09

#	โฟลเดอร์	รับผิดชอบเรื่องเดียว	บรรทัด	UC / FR
	dashboard			· FR-07
8	modules/alert	ตรวจเงื่อนไขสต็อกต่ำและคิวค้าง แล้วสร้างการแจ้งเตือน	115	UC-11 · FR-08
9	modules/audit-log	บันทึกประวัติและค้นหาย้อนหลัง	121	UC-10 · FR-09
10	modules/print-client	ส่งใบเสร็จไปให้บริการพิมพ์	45	UC-03 · FR-03

2.2 ส่วนสนับสนุน 3 ตัว (ตารางที่ 3.14) และไฟล์ประกอบระบบ

ไฟล์	หน้าที่	บรรทัด
config/db.js	ที่เดียวที่รู้ว่าฐานข้อมูลอยู่ไหน ให้ pool และ withTransaction แก่ทุกโมดูล เลือกฐานข้อมูลทดสอบอัตโนมัติเมื่อรันชุดทดสอบ	56
middleware/auth.js	ด่านตรวจหน้าทุกเส้นทาง: verifyToken (เป็นใคร) และ requireRole (มีสิทธิ์ไหม)	59
lib/promptpay.js	แปลงหมายเลขพร้อมเพย์ของร้านกับยอดเงินเป็นสตริงรหัส QR ไม่ยุ่งกับฐานข้อมูลหรือคำขอ	72
app.js	ประกาศเส้นทางบริการทั้ง 19 เส้นทางพร้อมสิทธิ์ของแต่ละเส้นทาง — เป็นตารางที่ 3.18 ในรูปโค้ด	75
server.js	เปิดพอร์ต ผูกช่องทางค้างเข้ากับเซิร์ฟเวอร์ และเริ่มตัวจับเวลาแจ้งเตือน	20

รวมหลังบ้านประมาณ 1,000 บรรทัดที่เขียนเอง ไม่นับชุดทดสอบและไลบรารีภายนอก

3. เส้นทางบริการและการตรวจสิทธิ์

เส้นทางทั้ง 19 เส้นทางประกาศอยู่ใน app.js ไฟล์เดียว เรียงตามระดับสิทธิ์จากเปิดกว้างไปเข้มงวด คือเส้นทางสาธารณะ เส้นทางที่ทุกบทบาทที่เข้าสู่ระบบเรียกได้ เส้นทางของพนักงานขาย เส้นทางของพนักงานคลัง และเส้นทางของผู้จัดการ เหตุผลที่รวมไว้ที่เดียวคือ ถ้ากระจายการตรวจสิทธิ์ไปอยู่ในแต่ละโมดูล จะตอบคำถามว่า “บทบาทนี้ทำอะไรได้บ้าง” ไม่ได้โดยไม่เปิดอ่านทุกไฟล์

ลำดับการทำงานของทุกคำขอเหมือนกันหมด คือ verifyToken → requireRole → ตัวจัดการของโมดูล ตัวอย่างการประกาศ

```
app.post('/api/orders', requireRole('sales', 'manager'), order.create);
app.post('/api/orders/:id/dispatch', requireRole('warehouse', 'manager'), dispatch.confirm);
app.get('/api/audit-logs', requireRole('manager'), auditLog.list);
```

กลุ่มเส้นทาง	จำนวน	ใครเรียกได้
ตรวจสุขภาพระบบและเข้าสู่ระบบ	2	ไม่ต้องเข้าสู่ระบบ
ข้อมูลผู้ใช้ปัจจุบัน ค้นหาสินค้า ดูคำสั่งซื้อ ช่องทางสำรอง	5	ทุกบทบาทที่เข้าสู่ระบบแล้ว
คำนวณยอดและสร้างคำสั่งซื้อ	2	พนักงานขาย ผู้จัดการ

กลุ่มเส้นทาง	จำนวน	ใครเรียกได้
คิว เปลี่ยนสถานะ ยืนยันจ่าย แดชบอร์ดปฏิบัติการ อ่านการแจ้งเตือน	5	พนักงานคลังสินค้า ผู้จัดการ
จัดการสินค้า ปรับสต็อก แดชบอร์ดบริหาร ทำเครื่องหมายอ่าน ประวัติ	6	ผู้จัดการ

การปฏิเสธแยกเป็นสองกรณีอย่างชัดเจน คือ **401** เมื่อไม่รู้ว่าเป็นใคร (ไม่แนบโทเคน โทเคนปลอม หรือหมดอายุ) และ **403** เมื่อรู้ว่าเป็นใครแต่บทบาทนั้นไม่มีสิทธิ์

4. กติกาการเขียนที่ยึดทั้งระบบ

กติกา	เหตุผล
หนึ่งโมดูลรับผิดชอบเรื่องเดียว และเขียนไว้ที่หัวไฟล์	เปิดไฟล์แล้วรู้ทันทีว่าโค้ดนี้ควรอยู่ที่นี่หรือไม่ (หัวข้อ 2.1.1)
โมดูลไม่เปิดการเชื่อมต่อฐานข้อมูลเอง ใช้ pool หรือ withTransaction จาก config/db.js	การตั้งค่าอยู่ที่เดียว และชุดทดสอบเปลี่ยนไปใช้ฐานข้อมูลทดสอบได้โดยไม่แตะโมดูล
เงื่อนไขที่กันข้อผิดพลาดต้องอยู่ในคำสั่ง SQL เดียวกับการเขียน ไม่ใช่อ่านมาตรวจในโปรแกรมแล้วค่อยเขียน	ระหว่างสองขั้นตอนนั้น คำขออื่นแทรกเข้ามาได้ (หัวข้อ 2.1.3)
ผลพลอยได้ (พิมพ์ใบเสร็จ ผลักเหตุการณ์ ตรวจแจ้งเตือน) ทำหลังธุรกรรมยืนยันแล้วเท่านั้น	ถ้าทำระหว่างธุรกรรม แล้วธุรกรรมถูกยกเลิก หน้าจอจะเห็นข้อมูลที่ไม่มีอยู่จริง
ความล้มเหลวที่คาดไว้ (สต็อกไม่พอ จ่ายซ้ำ) ไม่ถือเป็นข้อผิดพลาดของระบบ	แยกจากข้อผิดพลาดจริง ทำให้ตอบผู้ใช้ด้วยข้อความที่เข้าใจได้และไม่กลบปัญหาของจริง
ทุกการกระทำสำคัญบันทึกประวัติในธุรกรรมเดียวกับงานที่เกิด	ถ้าบันทึกแยก อาจมีงานที่สำเร็จแต่ไม่มีประวัติ หรือมีประวัติแต่งานถูกยกเลิก

5. กลไกสำคัญ 6 เรื่อง

5.1 ธุรกรรม — withTransaction

ผู้เรียกส่งงานเข้าไปเป็นฟังก์ชันเดียว ถ้าทำงานจนจบระบบจะยืนยันธุรกรรม (COMMIT) ถ้าโยนข้อผิดพลาดจะย้อนกลับ (ROLLBACK) ทั้งหมดแล้วโยนต่อ ผู้เรียกจึงไม่ต้องจำว่าต้องย้อนกลับเอง ซึ่งเป็นจุดที่มักลืมและทำให้เหลือข้อมูลค้างครึ่งทาง

5.2 ตัดสต็อกแบบอะตอมมิก — 7 ขั้นของหัวข้อ 3.7.3

เป็นกลไกที่สำคัญที่สุด เพราะตอบทั้งปัญหาบิลเก่ารับของซ้ำ (EV-06) และยอดสต็อกคลาดเคลื่อน (EV-07)

ขั้น	สิ่งที่ทำ	อยู่ที่ไหน
1	เปิดธุรกรรม	modules/dispatch
2	เปลี่ยนสถานะคำสั่งซื้อ โดยมีเงื่อนไขสถานะเดิมอยู่ในคำสั่งเดียวกัน	modules/stock-deduction

ขั้น	สิ่งที่ทำ	อยู่ที่ไหน
3	หักยอดคงเหลือทีละรายการ โดยมีเงื่อนไขว่ายอดต้องพอ	modules/stock-deduction
4	บันทึกการเคลื่อนไหวสต็อกพร้อมยอดหลังเปลี่ยน	modules/stock-deduction
5	บันทึกประวัติการทำรายการ	modules/stock-deduction
6	ยืนยันธุรกรรม	modules/dispatch
7	ตรวจเงื่อนไขแจ้งเตือน นอกธุรกรรม	modules/dispatch

คำสั่งสองบรรทัดนี้คือหัวใจของการกันซ้ำและกันติดลบ

```
UPDATE orders SET status = 'delivered', dispatched_by = ?, dispatched_at = NOW()
  WHERE order_id = ? AND status IN ('awaiting_dispatch','picking');

UPDATE products SET stock_qty = stock_qty - ? WHERE product_id = ? AND stock_qty >= ?;
```

ถ้าเครื่องอื่นจ่ายไปก่อนแล้ว คำสั่งแรกจะไม่ตรงเงื่อนไข จำนวนแถวที่ถูกแก้เป็นศูนย์ ระบบจึงยกเลิกทั้งธุรกรรมและตอบว่า “ถูกจ่ายไปแล้ว” (409) ถ้าสต็อกไม่พอ คำสั่งที่สองจะไม่ตรงเงื่อนไข ระบบยกเลิกทั้งธุรกรรมเช่นกัน สินค้ารายการที่หักไปแล้วในธุรกรรมนั้นย้อนกลับสู่ค่าเดิมทั้งหมด

อีกจุดที่ตั้งใจออกแบบคือการอ่านรายการสินค้าด้วย ORDER BY product_id เสมอ ทำให้ทุกธุรกรรมล็อกแถวตามลำดับเดียวกัน จึงไม่เกิดการล็อกไขว้กันจนค้าง (deadlock) ระหว่างสองคำขอที่มีสินค้าชุดเดียวกันแต่คนละลำดับ

5.3 เลขที่คำสั่งซื้อไม่ซ้ำ

รูปแบบ ORD-YYYYMMDD-NNNN นับต่อภายในวัน ระบบอ่านเลขล่าสุดของวันด้วยการล็อกแถว (FOR UPDATE) เครื่องที่ยืนยันพร้อมกันจึงต้องรอคิวกัน และมีการป้องกันอีกสองชั้น คือฐานข้อมูลกำหนดให้เลขที่ห้ามซ้ำ และถ้าชนกันจริงหรือธุรกรรมชนกันจนค้าง ระบบจะลองใหม่ทั้งธุรกรรมโดยอัตโนมัติพร้อมหน่วงเวลาแบบสุ่ม ผู้ใช้ไม่รับรู้

5.4 ช่องทางค้างสำหรับคิวเรียลไทม์

ช่องทางหลักคือ WebSocket ที่ /ws ช่องทางสำรองคือ Server-Sent Events ที่ /api/events (อุปกรณ์เครือข่ายบางรุ่นยกระดับการเชื่อมต่อเป็น WebSocket ไม่ได้) เหตุการณ์ที่ผลักมีสี่ชนิด คือ order.created, order.status, order.dispatched และ notification.created ทุกเหตุการณ์ผลักหลังธุรกรรมยืนยันแล้วเท่านั้น และหน้าจอใช้เหตุการณ์เป็นเพียง “สัญญาณให้ไปโหลดใหม่” ไม่ได้ประกอบรายการจากเหตุการณ์เอง ถ้าการเชื่อมต่อหลุดไปช่วงหนึ่ง เมื่อกลับมาแล้วโหลดใหม่ข้อมูลจึงครบเสมอ

5.5 การแจ้งเตือนเชิงรุก

สินค้าใกล้หมดตรวจหลังทุกครั้งที่สต็อกเปลี่ยนสำเร็จ (ตามกฎ BR-05) ส่วนคิวค้างตรวจด้วยตัวจับเวลาทุก 60 วินาที โดยใช้เกณฑ์ 30 นาทีซึ่งตั้งค่าได้โดยไม่ต้องแก้โค้ด ระบบไม่สร้างการแจ้งเตือนซ้ำขณะที่ใบเดิมของสิ่งเดียวกันยังไม่ถูกอ่าน เพื่อไม่ให้ผู้จัดการได้รับข้อความเดิมทุกครั้งที่จ่ายสินค้ารายการนั้น

5.6 ประวัติการทำรายการ

โมดูลประวัติเปิดฟังก์ชัน record(conn, ...) ที่รับการเชื่อมต่อของผู้เรียกเข้ามา จึงเขียนอยู่ในธุรกรรมเดียวกับงานที่เกิด ถ้างานถูกยกเลิก ประวัติก็หายไปด้วย รายการการกระทำถูกกำหนดไว้เป็นรายการปิด 10 ชนิด (เช่น LOGIN, CREATE_ORDER, CONFIRM_DISPATCH, ADJUST_STOCK) เพื่อให้ค้นหาตามประเภทได้และไม่มีใครเขียนชื่อการกระทำใหม่ตามใจ เวลาที่ค้นหา ระบบจะแปลงรหัสข้อมูลที่ถูกกระทำให้เป็นข้อความที่อ่านออก เช่น แสดงเลขที่คำสั่งซื้อพร้อมสรุปรายการสินค้าแทนที่จะแสดงเพียงหมายเลข

6. ฐานข้อมูลที่หลังบ้านพึ่งพา

ระบบเลือกให้ฐานข้อมูลเป็นผู้บังคับกฎที่สำคัญที่สุดด้วยตัวเอง เพราะกฎที่อยู่ในโปรแกรมอย่างเดียวจะหายไปทันทีที่มีใครเขียนข้อมูลด้วยช่องทางอื่น

ตาราง	เก็บอะไร	กฎที่ฐานข้อมูลบังคับเอง
users	ผู้ใช้ รหัสผ่านที่เข้ารหัส บทบาท สถานะเปิด/ปิด	ชื่อผู้ใช้ห้ามซ้ำ · บทบาทจำกัดสามค่า
products	สินค้า หมวดหมู่ ราคา หน่วยนับ ยอดคงเหลือ จุดสั่งซื้อเพิ่ม	รหัสสินค้าห้ามซ้ำ · ยอดคงเหลือห้ามติดลบ
orders	คำสั่งซื้อ เลขที่ ยอดรวม สถานะ ผู้สร้าง เวลารับชำระ ผู้จ่าย	เลขที่ห้ามซ้ำ · เวลารับชำระห้ามว่าง · สถานะจำกัดสี่ค่า
order_items	รายการสินค้าในคำสั่งซื้อ พร้อมราคา ณ เวลาขาย	จำนวนต้องมากกว่าศูนย์ · ผูกกับคำสั่งซื้อและสินค้าที่มีอยู่จริง
stock_movements	การเปลี่ยนแปลงยอดคงเหลือทุกครั้ง พร้อมยอดหลังเปลี่ยนและสาเหตุ	สาเหตุจำกัดสามค่า (จ่ายตามคำสั่งซื้อ ปรับด้วยมือ รับเข้า)
notifications	การแจ้งเตือนสินค้าใกล้หมดและคิวค้าง	ชนิดจำกัดสองค่า
audit_logs	ประวัติเหตุการณ์ ผู้กระทำ ไอพี เวลา	ไม่มีเส้นทางใดในระบบที่แก้หรือลบ

7. การพิมพ์ใบเสร็จ — ทำไมต้องแยกออกไป

เครื่องพิมพ์ความร้อน Epson TM-T82 ต่อกับเครื่องคอมพิวเตอร์หน้าร้านด้วยสาย USB ซอฟต์แวร์ที่สั่งงานเครื่องพิมพ์จึงต้องทำงานบนเครื่องนั้นเสมอ ในขณะที่หลังบ้านย้ายไปทำงานที่ใดก็ได้ ด้วยเหตุนี้บริการพิมพ์จึงเป็นหน่วยติดตั้งแยกต่างหาก (ตารางที่ 3.17) หลังบ้านรู้จักเพียงว่า “ส่งข้อมูลใบเสร็จไปแล้วได้ผลกลับมา” ไม่รู้จักยี่ห้อหรือชุดคำสั่งของเครื่องพิมพ์ ถ้าร้านเปลี่ยนเครื่องพิมพ์ จึงแก้ที่บริการพิมพ์ที่เดียว

สัญญาที่สำคัญคือ ฟังก์ชันส่งพิมพ์ **ไม่โยนข้อผิดพลาดเด็ดขาด** แต่คืนค่าว่าสำเร็จหรือไม่สำเร็จพร้อมสาเหตุเสมอ เพราะข้อกำหนดกำหนดไว้ว่าพิมพ์ไม่สำเร็จแล้วการขายต้องยังสำเร็จ (AC-05) หน้าจอจะแสดงคำเตือนให้พนักงานทราบแทน และหลังบ้านไม่ส่งซ้ำอัตโนมัติเพื่อไม่ให้ได้ใบเสร็จสองใบ

8. การทดสอบ

ชุดทดสอบเป็นหลักฐานว่ากลไกข้างต้นทำงานจริง ไม่ใช่เพียงคำอธิบาย แบ่งเป็นสองชนิด คือการทดสอบหน่วย (ไม่ใช้ฐานข้อมูล) และการทดสอบเชิงบูรณาการ (ใช้ฐานข้อมูลทดสอบแยกชื่อ soml_new_test ซึ่งสร้างใหม่ทุกครั้งที่รัน)

ชุดทดสอบ	จำนวน	พิสูจน์อะไร
promptpay · escpos · print-service · print-client	34	สตริงรหัส QR ถูกต้องตามรูปแบบ · ใบเสร็จแปลงเป็นชุดคำสั่งเครื่องพิมพ์ถูกต้อง · พิมพ์ล้มแล้วไม่โยนข้อผิดพลาด
auth · auth-middleware	20	ปฏิเสธคำขอที่ไม่มีสิทธิ์ทุกรูปแบบ (401/403) และข้อความเข้าสู่ระบบไม่สำเร็จเหมือนกันทุกกรณี
product	16	แก้ข้อมูลสินค้าและจุดสั่งซื้อเพิ่ม · ปรับสต็อกจนติดลบถูกปฏิเสธ · มีบันทึกการเคลื่อนไหวและประวัติ
order	16	ยอดคำนวณจากราคาในฐานข้อมูล · สร้าง 20 คำสั่งซื้อพร้อมกันแล้วเลขไม่ซ้ำ · บริการพิมพ์ปิดอยู่แล้วคำสั่งซื้อยังสำเร็จ
queue	11	ทุกลูกศรของแผนภาพสถานะ รวมทั้งกรณีที่ต้องถูกปฏิเสธ · ทั้ง WebSocket และ SSE ได้รับเหตุการณ์จริง
dispatch	11	จ่ายพร้อมกันสำเร็จเพียงรายการเดียว · สต็อกไม่พอแล้วย้อนกลับทั้งธุรกรรม · 10 ธุรกรรมพร้อมกันไม่เกิดสภาวะแข่งขัน
alert-dashboard · audit-log · db	22	แจ้งเตือนเกิดเองและไม่ซ้ำ · ตัวเลขแดชบอร์ดตรงกับผลรวมจากฐานข้อมูล · ประวัติค้นหาตามเงื่อนไขได้
รวม	130	ผ่านทั้งหมด ใช้เวลารันประมาณ 4 วินาที

ความครอบคลุมของโค้ดโดยรวมอยู่ที่ประมาณร้อยละ 88 ของคำสั่ง โดยโมดูลตัดสต็อกซึ่งเป็นส่วนสำคัญที่สุดครอบคลุมร้อยละ 100

9. สิ่งที่ยังไม่ได้ทำและข้อจำกัดที่ทราบ

ประเด็น	สถานะปัจจุบัน	สิ่งที่ต้องทำต่อ
HTTPS	ระหว่างพัฒนาใช้ HTTP ในเครือข่ายภายใน	สร้างใบรับรองสำหรับเครือข่ายร้านและตั้งค่าส่งต่อ http ⟶ https ก่อนติดตั้งจริง
เครื่องพิมพ์จริง	ทดสอบกับบริการพิมพ์จำลองและไฟล์ผลลัพธ์แล้ว ยังไม่ได้ต่อเครื่อง TM-T82 ของร้าน	ต่อผ่าน USB พิมพ์ใบเสร็จจริง ตรวจภาษาไทยและจับเวลาเทียบเกณฑ์ 5 วินาที
ภาระงานพร้อมกัน	ทดสอบธุรกรรมพร้อมกันแล้ว แต่ยังไม่ได้ทดสอบ 10 เซสชันต่อเนื่องตาม NFR-06	จำลองผู้ใช้ 10 คนจากสามบทบาทใช้งานพร้อมกัน
ยอดสต็อกตั้งต้น	นำเข้าจากไฟล์สต็อกของร้าน 783 รายการใน 16 หมวดหมู่	นับของจริงเทียบกับระบบก่อนเริ่มใช้งาน
คู่มือติดตั้ง	ยังไม่ได้เขียน	เขียนแล้วให้ผู้ที่ไม่ได้ร่วมพัฒนาติดตั้งตาม เพื่อทวนสอบ

ประเด็น	สถานะปัจจุบัน	สิ่งที่ต้องทำต่อ
		NFR-09

10. คำถามที่พบบ่อยเกี่ยวกับหลังบ้าน

ถาม: ทำไมไม่เก็บตะกร้าสินค้าไว้ในฐานข้อมูล ตอบ: เพราะแผนภาพสถานะกำหนดให้สถานะแรกของคำสั่งซื้อคือรอจ่ายสินค้า และคอลัมน์เวลารับชำระห้ามว่าง การเขียนแถวก่อนรับเงินจะขัดกับทั้งสองข้อ ผลพลอยได้คือฐานข้อมูลไม่มีคำสั่งซื้อค้างจากลูกค้าที่เปลี่ยนใจ

ถาม: ถ้าลูกค้าโอนเงินแล้วแต่พนักงานยังไม่กดยืนยัน ข้อมูลอยู่ที่ไหน ตอบ: ยังไม่มีข้อมูลใดในระบบ เพราะระบบไม่ได้เชื่อมกับธนาคาร จึงไม่รู้ว่าเงินเข้าแล้ว การยืนยันรับชำระเป็นการกระทำของพนักงานหลังตรวจจากแอปพลิเคชันธนาคารด้วยตนเอง ข้อเท็จจริงนี้ถูกระบุไว้เป็นข้อสมมติของโครงงาน

ถาม: ถ้าสองคนกดยืนยันจ่ายสินค้าใบเดียวกันพร้อมกันจะเป็นอย่างไร ตอบ: สำเร็จเพียงคนเดียว เพราะเงื่อนไขสถานะเดิมอยู่ในคำสั่งปรับปรุงข้อมูลเดียวกัน คนที่สองจะได้จำนวนแถวที่ถูกแก้เป็นศูนย์ ระบบยกเลิกธุรกรรมและตอบว่าถูกจ่ายไปแล้ว สต็อกจึงถูกหักครั้งเดียว มีชุดทดสอบยิงพร้อมกัน 10 ธุรกรรมยืนยันเรื่องนี้

ถาม: ทำไมต้องมีทั้ง WebSocket และ SSE ตอบ: WebSocket เป็นช่องทางหลัก แต่อุปกรณ์เครือข่ายบางรุ่นยกระดับการเชื่อมต่อไม่ได้ จึงเตรียม SSE ไว้เป็นช่องทางสำรองที่ได้รับเหตุการณ์ชุดเดียวกัน หน้าจอสลับเองโดยผู้ใช้ไม่ต้องทำอะไร

ถาม: ถ้าเปลี่ยนเครื่องพิมพ์ ต้องแก้ตรงไหนบ้าง ตอบ: แก้ที่บริการพิมพ์ที่เดียว เพราะหลังบ้านรู้จักเพียงข้อมูลใบเสร็จและผลลัพธ์ ไม่รู้จักชุดคำสั่งของเครื่องพิมพ์

ถาม: ประวัติการทำรายการแก้ไขได้ไหม ตอบ: ไม่ได้ ระบบไม่มีเส้นทางบริการสำหรับแก้ไขหรือลบ มีเพียงการเพิ่มและการค้นหา และการเพิ่มเกิดในธุรกรรมเดียวกับงานที่เกิด